Learning for Planning

WASP Relational Learning Course 2026

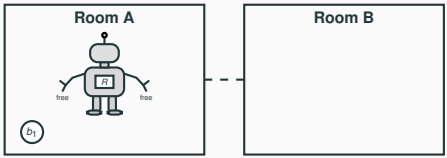
Dominik Drexler

May 18, 2026

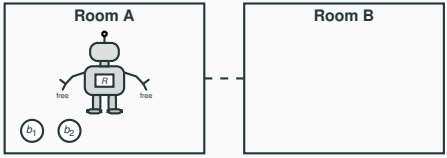
Postdoc
Machine Reasoning Lab
Linköping University

Motivation & Preview

Motivating Running Example: The Gripper Domain



Initial state with 1 ball



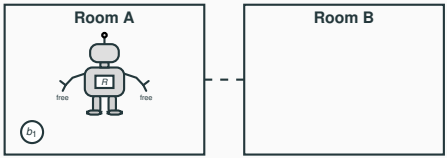
Initial state with 2 balls

and so on, for arbitrarily many balls

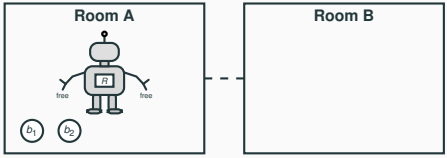
Generalized planning problem

- Available actions: move, pick, and drop.
- The number of balls is not fixed.
- A solution should describe one efficient strategy for every task in the class.
- STRIPS planning is PSPACE-complete in general, but many structured task classes are much easier.

Motivating Running Example: The Gripper Domain



Initial state with 1 ball



Initial state with 2 balls

and so on, for arbitrarily many balls

Generalized planning problem

- Available actions: move, pick, and drop.
- The number of balls is not fixed.
- A solution should describe one efficient strategy for every task in the class.
- STRIPS planning is PSPACE-complete in general, but many structured task classes are much easier.
- In this lecture: How to represent and learn solutions for such problems, among other things

Preview: Generalized Policy for the Gripper Domain (Francès, Bonet, and Geffner 2021)

State abstraction: features

$$
\begin{array}{rcll}
H & \equiv & \exists \text{carry}.\top & \text{; holding a ball?} \\
R_A & \equiv & \text{at-robby} \sqcap \{A\} & \text{; robot is in room A?} \\
n & \equiv & \text{ball} \sqcap \neg \exists \text{at}.\{B\} & \text{; balls not yet in room B}
\end{array}
$$

- These rules do not name individual balls; they describe how abstract features should change.

Temporal abstraction: policy rules

$$
\begin{array}{rcll}
r_1 & := & \{R_A, \neg H, n > 0\} \mapsto \{H\} & \text{; get hold of some ball in room A} \\
r_2 & := & \{R_A, H, n > 0\} \mapsto \{\neg R_A\} & \text{; move to room B} \\
r_3 & := & \{\neg R_A, H, n > 0\} \mapsto \{\neg H, n\downarrow\} & \text{; drop the carried ball in room B} \\
r_4 & := & \{\neg R_A, \neg H, n > 0\} \mapsto \{R_A\} & \text{; return to room A}
\end{array}
$$

Scope of This Lecture

- **Goal:** understand the ingredients that let us judge whether a relational learning approach can work, not survey all existing methods.
- We therefore focus on the basic underlying mathematics and well-established representation languages.
- Some technical details are included as reference material for later reading and project work.

Background

PDDL Recap: Domains

- A first-order planning domain $\mathcal{D}$ consists of:
  - relation symbols $\mathcal{R}$
  - first-order action schemas $\mathcal{A}$

Example: Gripper predicates

```
(:predicates
    (room ?r)
    (ball ?b)
    (gripper ?g)
    (at-robby ?r)
    (at ?b ?r)
    (free ?g)
    (carry ?b ?g))
```

PDDL Recap: Action Schemas

- Action schemas describe how the state can change.

Example: Moving between rooms

```
(:action move
 :parameters (?from ?to)
 :precondition (and
    (room ?from)
    (room ?to)
    (at-robby ?from))
 :effect (and
    (at-robby ?to)
    (not (at-robby ?from))))
```

PDDL Recap: Task Instances

- A task instance provides objects, initial state atoms, and goal atoms.

Example: Gripper task with one ball

```
(:objects
    rooma roomb - room
    ball1 - ball
    left right - gripper)

(:init
    (at-robby rooma)
    (at ball1 rooma)
    (free left)
    (free right))

(:goal (and
    (at ball1 roomb)))
```

Definition (Classical planning task)

A classical planning task over a first-order domain $\mathcal{D} = \langle \mathcal{R}, \mathcal{A} \rangle$ is a tuple

$$P = \langle \mathcal{D}, O_P, I_P, \gamma_P \rangle,$$

where O_P is a finite set of objects, I_P is an initial state, and γ_P is a goal condition.

- Grounding P induces an ordinary propositional planning task (e.g., STRIPS).

Definition (Classical plan)

A solution (or plan) for P is a finite sequence $\langle a_1, \ldots, a_m \rangle$ of applicable ground actions from P that transforms I_P into a state satisfying γ_P.

- A plan is specific to the objects, atoms, and actions of the task P.

State Models

Definition (State model)

A classical planning task $P = \langle \mathcal{D}, O_P, I_P, \gamma_P \rangle$ with $\mathcal{D} = \langle \mathcal{R}, \mathcal{A} \rangle$ induces a state model

$$\mathcal{S}_P = \langle S_P, A_P, T_P, I_P, G_P \rangle,$$

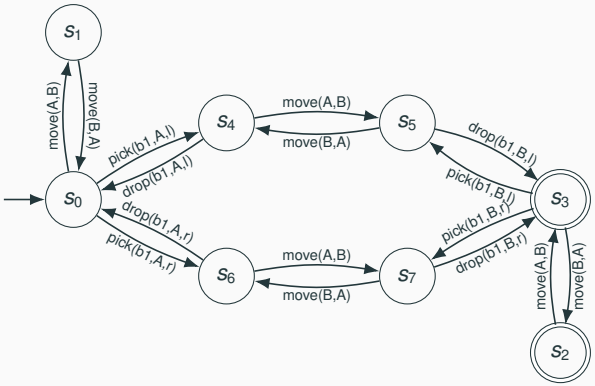
where

$$\begin{aligned} S_P &: \text{states in } P, \\ A_P &: \text{ground actions from } \mathcal{A} \text{ and } O_P, \\ T_P \subseteq S_P \times S_P &: \text{transition relation}, \\ I_P \in S_P &: \text{initial state}, \\ G_P \subseteq S_P &: \text{goal states}. \end{aligned}$$

- $(s, s') \in T_P$ iff some applicable ground action leads from state s to state s'.
- Planning means finding an action sequence that follows transitions from I_P to some state in G_P.

Example: Gripper State Model with One Ball

- Nodes are states; directed edges are applicable ground actions.
- A solution: pick(b1,A,l), move(A,B), drop(b1,B,l)



The Scope of Heuristic Guidance

- A **heuristic** is a function $h : S \to \mathbb{N}_0 \cup \{\infty\}$ used to guide search (Module 1).
- Heuristics can be designed or learned for different scopes.

Scope	Notation
General	$h : S \to \mathbb{N}_0 \cup \{\infty\}$
Task class $\mathcal{Q} = \{P_1, P_2, \ldots\}$	$h_{\mathcal{Q}} : {S_{\mathcal{Q}}}^1 \to \mathbb{N}_0 \cup \{\infty\}$
Single task P	$h_P : S_P \to \mathbb{N}_0 \cup \{\infty\}$

- Domain-independent planning sits at the **general scope** (Module 1).
- Some Reinforcement Learning (RL) approaches sit at the **single task scope** (e.g., Alpha Go).
- Generalized heuristics sit in the **task class scope**: they are not task-specific, but they may exploit the shared structure of a task class $\mathcal{Q}$.

[1] $S_{\mathcal{Q}} = \biguplus_{P \in \mathcal{Q}} S_P$.

Generalized Classical Planning

Definition (Generalized classical planning problem)

A generalized classical planning problem is a class

$$\mathcal{Q} = \{P_1, P_2, \ldots\}$$

of classical planning tasks over the same first-order domain $\mathcal{D}$.

Definition (Generalized solution)

A solution for $\mathcal{Q}$ is an algorithm that, for every task $P \in \mathcal{Q}$, computes a plan that reaches the goal of P in time polynomially bounded in the size of P.

- The solution must be object-independent: it cannot refer to task-specific objects, atoms, or actions.
- In other words, it may only use information that is meaningful across the whole class $\mathcal{Q}$: the shared predicate vocabulary (= **relations**) and domain structure.
- This is a strong notion: auxiliary objectives, e.g., dead-end detection, can speed up planning without guaranteeing polynomial-time plan generation (tackled in the project).

Relational Learning in AI Planning

State Abstractions: Feature Languages

- If we restrict attention to a class $\mathcal{Q}$, then all tasks in $\mathcal{Q}$ share the same domain predicates.
- What changes from task to task are the objects, ground atoms, ground actions, and state models.
- Feature languages give us object-independent descriptions of states
- They build the foundation for temporal abstractions: generalized policies π and heuristics $h_{\mathcal{Q}}$.

State Abstractions: Feature Languages

State abstraction: features

$\#A \equiv$ how many balls are in room A,

$\#G \equiv$ how many balls are in the gripper,

$R_A \equiv$ is the robot in room A?.

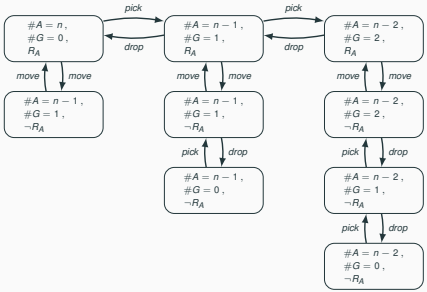


Figure 1: An abstraction of a Gripper task with n balls.

States as Relational Structures

Definition (Relational structure)

A relational structure over $\mathcal{R}$ is $\mathcal{I} = \left(\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}}\right)$, where $\Delta^{\mathcal{I}}$ is a set of objects and each k-ary $R \in \mathcal{R}$ is interpreted as $R^{\mathcal{I}} \subseteq (\Delta^{\mathcal{I}})^k$.

- A state s of task P induces $\mathcal{I}_s = \left(O_P, \cdot^{\mathcal{I}_s}\right)$: predicates are interpreted by the tuples true in s.

Example (Gripper initial state s as a relational structure)

Planning state s	**Relational structure $\mathcal{I}^s$**
$O_P = \{\text{rooma}, \text{roomb}, \text{ball1}, \text{left}, \text{right}\}$	$\Delta^{\mathcal{I}_s} = O_P$
room(rooma), room(roomb)	$\text{room}^{\mathcal{I}_s} = \{\text{rooma}, \text{roomb}\}$
ball(ball1)	$\text{ball}^{\mathcal{I}_s} = \{\text{ball1}\}$
gripper(left), gripper(right)	$\text{gripper}^{\mathcal{I}_s} = \{\text{left}, \text{right}\}$
at-robby(rooma)	$\text{at-robby}^{\mathcal{I}_s} = \{\text{rooma}\}$
at(ball1, rooma)	$\text{at}^{\mathcal{I}_s} = \{(\text{ball1}, \text{rooma})\}$
free(left), free(right)	$\text{free}^{\mathcal{I}_s} = \{\text{left}, \text{right}\}$
	$\text{carry}^{\mathcal{I}_s} = \emptyset$

$$R(\bar{o}) \in s \iff \bar{o} \in R^{\mathcal{I}_s}.$$

- This connects planning states to standard tools for finite relational structures.

States as Relational Structures

- One standard notion we get for free is invariance under renaming objects.

Definition (Isomorphism)

Two relational structures (states) $\mathcal{I}$ and $\mathcal{J}$ over the same relations $\mathcal{R}$ are isomorphic, denoted by $\mathcal{I} \cong \mathcal{J}$, if there is a relation-preserving bijection $h : \Delta^{\mathcal{I}} \to \Delta^{\mathcal{J}}$ such that, for every k-ary $R \in \mathcal{R}$,

$$(o_1, \ldots, o_k) \in R^{\mathcal{I}} \Leftrightarrow (h(o_1), \ldots, h(o_k)) \in R^{\mathcal{J}}.$$

Example (Object names should not matter)

Two Gripper states that only differ by renaming ball1 to ball2 are isomorphic. A feature language should therefore assign them the same value:

$$\mathcal{I}_s \cong \mathcal{I}_{s'} \quad \Longrightarrow \quad f(s) = f(s').$$

- Isomorphism-invariant feature languages are therefore suitable candidates: first-order logic, description logics, Weisfeiler-Leman, graph neural networks, ...
- These languages **cannot** distinguish between isomorphic relational structures by design.

Feature Language 1: Description Logics

Description Logics (DL)

- Description logics are knowledge-representation languages for describing objects in relational structures (Baader et al. 2003).
- Concept names are unary relations; role names are binary relations in the underlying structure.
- This gives DLs their basic limitation: they operate directly with unary and binary relations only.
- A concept expression C denotes a set of objects $C^{\mathcal{I}_s}$ and induces a numerical feature by counting: $n_C(s) = |C^{\mathcal{I}_s}|$. A Boolean feature $B_C(s)$ is true iff $n_C(s) > 0$, and false otherwise.

Example (Balls in the target room)

Representation	Expression
Goal-aware first-order logic	$\text{ball}(b) \wedge \exists r, b'.\text{at}_G(b', r) \wedge \text{at}(b, r)$
Goal-aware description logic	$\exists \text{at}.\exists \text{at}_G{}^{-1}.\top$
Description logic with nominals	$\exists \text{at}.\{B\}$

Description Logic $\mathcal{AL}$: Syntax and Semantics

- $\mathcal{AL}$, the *Attributive Language*, builds concept expressions over a relational structure $\mathcal{I}$.
- The grammar for concept expressions is:

$$C, D ::= \top \mid \bot \mid A \mid \neg A \mid C \sqcap D \mid \forall R.C \mid \exists R.\top, \qquad A \in N_C,\ R \in N_R.$$

Syntax	**Semantics**
$\top$	$\Delta^{\mathcal{I}}$
$\bot$	$\emptyset$
A	$A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$
$\neg A$	$\Delta^{\mathcal{I}} \setminus A^{\mathcal{I}}$
$C \sqcap D$	$C^{\mathcal{I}} \cap D^{\mathcal{I}}$
$\forall R.C$	$\{x \in \Delta^{\mathcal{I}} \mid \forall y.(x, y) \in R^{\mathcal{I}} \rightarrow y \in C^{\mathcal{I}}\}$
$\exists R.\top$	$\{x \in \Delta^{\mathcal{I}} \mid \exists y.(x, y) \in R^{\mathcal{I}}\}$

- Takeaway: $\mathcal{AL}$ shows the semantic pattern, but useful Gripper features need extensions such as nominals or goal-derived predicates; balancing expressivity is a challenge.

Description Logics: Obtaining Feature Sets $\mathcal{F}$

- **Concept learning:** learn useful description-logic concept expressions from labeled examples (Module 2).

$\rightarrow$ Approaches exist outside generalized planning (e.g. Demir and Ngonga Ngomo 2023), but have not yet been explored in generalized planning.

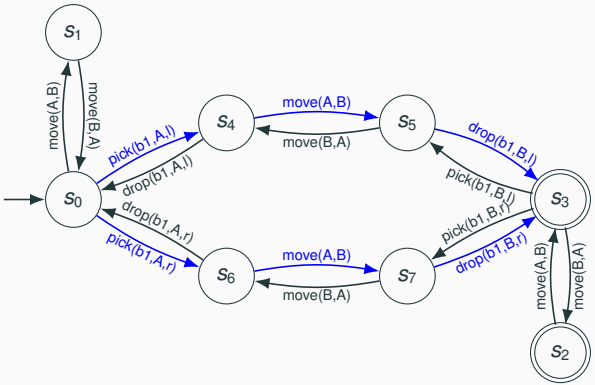
- **Exhaustive enumeration:** generate concepts up to a bounded size from grammar specification.

- **Semantic pruning:** discard concepts that behave identically on the available relational structures.

Temporal Abstraction 1: Generalized Policies

Generalized Policies (Francès, Bonet, and Geffner 2021)

- General policies capture what to do in each state to reach the goal.
- A **policy** π for a class of tasks $\mathcal{Q}$ is a relation that, for every task $P \in \mathcal{Q}$, defines $\pi_P \subseteq T_P$.
- A policy π is a **generalized policy** for $\mathcal{Q}$ if, for every task $P \in \mathcal{Q}$, every way of following the policy reaches a goal state in polynomial time.



Preview: Generalized Policy for the Gripper Domain (Francès, Bonet, and Geffner 2021)

State abstraction: features

$$
\begin{array}{rcll}
H & \equiv & \exists \text{carry}.\top & \text{; holding a ball?} \\
R_A & \equiv & \text{at-robby} \sqcap \{A\} & \text{; robot is in room A?} \\
n & \equiv & \text{ball} \sqcap \neg \exists \text{at}.\{B\} & \text{; balls not yet in room B}
\end{array}
$$

Temporal abstraction: policy rules

$$
\begin{array}{rcll}
r_1 & := & \{R_A, \neg H, n > 0\} \mapsto \{H\} & \text{; get hold of some ball in room A} \\
r_2 & := & \{R_A, H, n > 0\} \mapsto \{\neg R_A\} & \text{; move to room B} \\
r_3 & := & \{\neg R_A, H, n > 0\} \mapsto \{\neg H, n\downarrow\} & \text{; drop the carried ball in room B} \\
r_4 & := & \{\neg R_A, \neg H, n > 0\} \mapsto \{R_A\} & \text{; return to room A}
\end{array}
$$

Learning Generalized Policies via Satisfiability (SAT)

Training Data

Tasks:	$\mathcal{Q}_{\text{train}} = \{P_i\}_{i=1}^{k} \subseteq \mathcal{Q}$
States:	$\mathcal{S}_{\text{train}} = \biguplus_{P \in \mathcal{Q}_{\text{train}}} \mathcal{S}_P$
Transitions:	$\mathcal{T}_{\text{train}} = \biguplus_{P \in \mathcal{Q}_{\text{train}}} \mathcal{T}_P$
Features:	$\mathcal{F}$ from a feature language

SAT Variables

Theory $\mathcal{T}(\mathcal{S}_{\text{train}}, \mathcal{T}_{\text{train}}, \mathcal{F})$.

$\textsc{Select}(f)$	feature f is used by the policy
$\textsc{Good}(s, s')$	transition (s, s') is π-compatible

Policy Constraints

1. Every solvable, non-goal state $s \in \mathcal{S}_{\text{train}}$ chooses at least one good transition:

$$\bigvee_{\{s' \mid (s,s') \in \mathcal{T}_{\text{train}}\}} \textsc{Good}(s, s')$$

2. Progress / acyclicity constraints prevent cycles among $\textsc{Good}$ transitions.

Feature Constraints

1. Every "good" transition $(s, s') \in \pi$ must be separable from every "bad" transition $(t, t') \notin \pi$ by a selected feature:

$$\textsc{Good}(s, s') \wedge \neg \textsc{Good}(t, t') \rightarrow \bigvee_{\substack{f \in \mathcal{F} \\ \triangle_f(s,s') \neq \triangle_f(t,t')}} \textsc{Select}(f)$$

where $\triangle_f(s, s')$ is the abstract transition observation over feature f, e.g.,

$$\{H\} \mapsto \{\neg H\} \quad \text{and} \quad \{\neg H\} \mapsto \{H\}$$

Compared to ILP: SAT jointly chooses useful transitions and useful features.
Occam's razor: Minimizing the sum of selected feature complexities often yields a policy that generalizes over $\mathcal{Q}$.

Alternative Feature Languages

- Description logics are not the only option for feature languages.
- Languages with related expressiveness include:
  - Two-variable first order logic with counting quantifiers (C_2)
  - Color refinement (1-dimensional Weisfeiler-Leman)
  - Aggregate-Combine-Readout Graph Neural Networks (ACR-GNNs)
- **Expressive power** measured by the ability to distinguish non-isomorphic relational structures.
- C_2 is sufficient for structurally simple classes of tasks.
- You can define general policies and heuristics over these languages as well.

Feature Language 2: Weisfeiler Leman

Graphs

- We can map relational structures into graphs
- Enables graph-based feature languages such as color refinement and graph neural networks.

Definition (Graph)

A graph is a tuple $G = (V, E)$, where V is a set of vertices and $E \subseteq V \times V$ is a set of edges.

Definition (Colored graph)

A colored graph is a tuple $G = (V, E, \lambda)$, where (V, E) is a graph and $\lambda : V \to \Sigma$ assigns a color to each vertex.

Relational Structures as Graphs

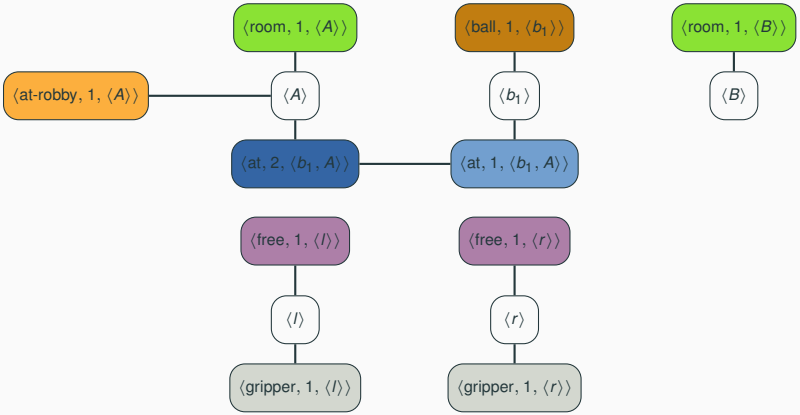


Figure 2: A planning state as a relational graph: objects are connected through colored predicate-position vertices.

Relational Structures as Graphs

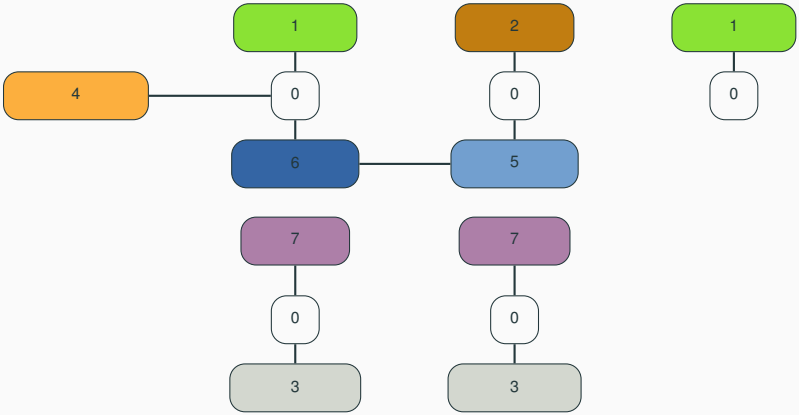


Figure 3: A planning state as a relational graph: objects are connected through colored predicate-position vertices.

Relational Structures as Graphs: Properties

Definition (Colored graph isomorphism)

Two colored graphs $G = (V, E, \lambda)$ and $G' = (V', E', \lambda')$ are isomorphic, denoted by $G \cong G'$, if there is an edge-preserving and vertex-color-preserving bijection $h : V \to V'$ such that

$$(u, v) \in E \Leftrightarrow (h(u), h(v)) \in E' \quad \text{for all } u, v \in V,$$
$$\lambda(x) = \lambda'(h(x)) \quad \text{for all } x \in V.$$

- Consider a function f that maps relational structures to colored graphs.
- **Isomorphism preserving:** for any two relational structures $\mathcal{I}$ and $\mathcal{J}$,

$$\mathcal{I} \cong \mathcal{J} \quad \Leftrightarrow \quad f(\mathcal{I}) \cong f(\mathcal{J}).$$

- If an encoding does not preserve isomorphism, then we may lose the ability to distinguish between non-isomorphic structures, which may reduce expressivity in the feature language.
- **Takeaway:** the choice of encoding is a design decision, but experimenting with an isomorphism-preserving encoding is often beneficial.

Color Refinement (1-Dimensional Weisfeiler-Leman)

► Color refinement (1-WL) is a graph partitioning and approximate graph isomorphism algorithm (Weisfeiler and Leman 1968).

► It takes as input a colored graph $G = (V, E, \lambda)$.

► In each round, the new color of a vertex is determined by its current color and the multiset of colors of its neighbors:

$$\lambda_{i+1}(v) = (\lambda_i(v), \{\{\lambda_i(w) \mid w \text{ is adjacent to } v\}\})$$

► Colors can be viewed as abstract identifiers, e.g., integers or structured objects; different identifiers mean different colors.

► If we view colors as integers, then standard classical ML techniques can learn strong planning heuristics from 1-WL features (Chen, Trevizan, and Thiébaux 2024).

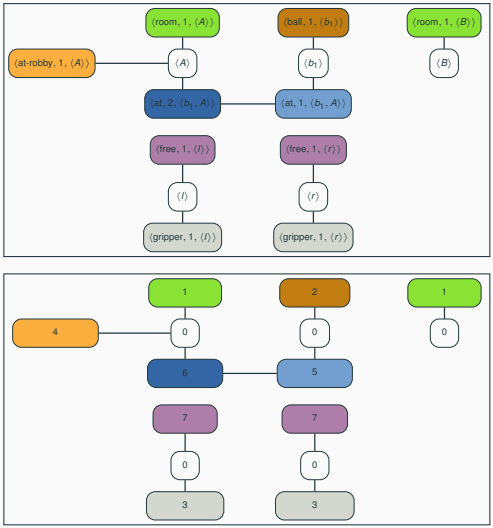
Color Refinement (1-Dimensional Weisfeiler-Leman): Example

- $\lambda_{i+1}(v) = (\lambda_i(v), \{\{\lambda_i(w) \mid w \in N(v)\}\})$.

vertices	$(\lambda_0(v), \{\{\lambda_0(w) \mid w \in N(v)\}\})$	$\lambda_1(v)$
$\langle A \rangle$	$(0, \{\{1, 4, 6\}\})$	0
$\langle B \rangle$	$(0, \{\{1\}\})$	1
$\langle b_1 \rangle$	$(0, \{\{2, 5\}\})$	2
$\langle l \rangle$	$(0, \{\{3, 7\}\})$	3
$\langle r \rangle$	$(0, \{\{3, 7\}\})$	3
$\langle \text{room}, 1, \langle A \rangle \rangle$	$(1, \{\{0\}\})$	4
$\langle \text{room}, 1, \langle B \rangle \rangle$	$(1, \{\{0\}\})$	4
$\langle \text{ball}, 1, \langle b_1 \rangle \rangle$	$(2, \{\{0\}\})$	5
$\langle \text{gripper}, 1, \langle l \rangle \rangle$	$(3, \{\{0\}\})$	6
$\langle \text{gripper}, 1, \langle r \rangle \rangle$	$(3, \{\{0\}\})$	6
$\langle \text{at-robby}, 1, \langle A \rangle \rangle$	$(4, \{\{0\}\})$	7
$\langle \text{at}, 1, \langle b_1, A \rangle \rangle$	$(5, \{\{0, 6\}\})$	8
$\langle \text{at}, 2, \langle b_1, A \rangle \rangle$	$(6, \{\{0, 5\}\})$	9
$\langle \text{free}, 1, \langle l \rangle \rangle$	$(7, \{\{0\}\})$	10
$\langle \text{free}, 1, \langle r \rangle \rangle$	$(7, \{\{0\}\})$	10

Table 1: First iteration of color refinement.

- Graph Neural Networks (GNNs) work similarly.



Color Refinement: A Limitation

► Color refinement does not distinguish all non-isomorphic graphs.

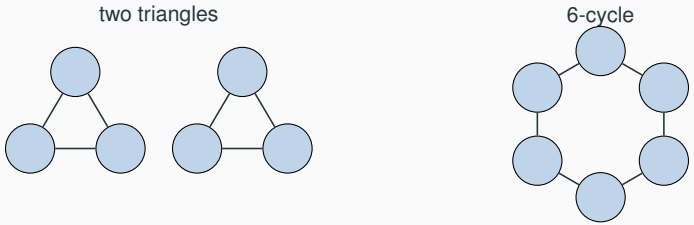


Figure 4: Color refinement cannot distinguish two triangles and a 6-cycle.

► In planning, the same issue can occur when two states require different heuristic or policy values but induce graphs with the same 1-WL colors.

Feature Language 3: Graph Neural Networks

Graph Neural Networks (Xu et al. 2018; Barceló et al. 2020)

Message passing as learned color refinement

$$\text{1-WL: } \lambda_{i+1}(v) = (\lambda_i(v), \{\{\lambda_i(w) : w \in N(v)\}\})$$

$$\text{ACR-GNN: } \mathbf{h}_{i+1}(v) = \mathrm{COM}_\theta(\mathbf{h}_i(v), \mathrm{AGG}_\theta\{\{\mathbf{h}_i(w) : w \in N(v)\}\}, \mathrm{READ}_\theta\{\{\mathbf{h}_i(w) : w \in V\}\})$$

- Both iteratively aggregate local neighborhood information.
- Message-passing GNNs use learned vector representations instead of symbolic colors.
- COM: MLP+sigmoid, AGG: sum/mean/max, READ: sum
- 1-WL and C^2 have the same distinguishing power on graphs / relational structures (Cai, Fürer, and Immerman 1992; Barceló et al. 2020; Grohe 2021), while ACR-GNN features can capture all C^2-definable features and add learned numerical readouts over 1-WL colors:

$$\text{1-WL} \equiv C^2 \subseteq \text{ACR-GNN features} \quad \text{(via global readout)}$$

Experimental Results: Expressivity Requirements

- **Language expressivity conflicts** are failures to distinguish non-isomorphic relational structures.
- This gap in expressivity often causes issues for learning general policies with R-GNNs + RL (Ståhlberg, Bonet, and Geffner 2023).

Domain	Coverage (%)	1-WL # conflicts
Delivery	100%	0
Gripper	100%	0
Logistics	36%	131
Grid	79%	42
Blocks	100%	50

Table 2: Learning general policies with R-GNNs + RL.

- Failures often explainable through the logical connection (Horcík and Sír 2024; Drexler et al. 2024)

Temporal Abstraction 2: Generalized Heuristics

Generalized Potential Heuristics (Francès, Corrêa, et al. 2019)

- Generalized potential heuristic: $h_{\mathcal{Q}}(s) = \sum_{f \in \mathcal{F}} w_f \cdot |f^{\mathcal{I}_s}|$.
- Descending and Dead-End avoiding: greedily following a state trajectory that decreases the heuristic leads to a goal in polynomial time.

Example (Gripper)

State abstraction: features

$C_1 \equiv$ Balls that are in a room different from the target room
$C_2 \equiv$ Balls carried by the robot
$C_3 \equiv$ Balls carried by some robot while robot is in the target room
$C_4 \equiv$ Robot in non-empty rooms other than the target room

- Generalized potential heuristic $h_{\mathcal{Q}}(s) = 8 \cdot |C_1^{\mathcal{I}_s}| + 4 \cdot |C_2^{\mathcal{I}_s}| - 2 \cdot |C_3^{\mathcal{I}_s}| - |C_4^{\mathcal{I}_s}|$

Summary

- In planning, **relational structures** describe states of the world in terms of objects, properties, and relations.
- **Feature languages** that are isomorphism-invariant allow generalization from smaller to larger tasks in $\mathcal{Q}$.
- There are many different feature languages, often related in their underlying logical expressivity.
- **Temporal abstraction** asks for reusable guidance: what should be done across a whole class of tasks $\mathcal{Q}$?
- The **propositional encoding** makes generalized policy learning explicit as constraints over features and transitions.

References

Baader, Franz, Diego Calvanese, Deborah L. McGuinness, Daniele Nardi, and Peter F. Patel-Schneider, eds. (2003). The Description Logic Handbook: Theory, Implementation and Applications. Cambridge University Press.

Barceló, Pablo, Egor Kostylev, Mikaël Monet, Jorge Pérez, Juan Reutter, and Juan-Pablo Silva (2020). **"The Logical Expressiveness of Graph Neural Networks".** In: *Proceedings of the Eighth International Conference on Learning Representations (ICLR 2020)*. OpenReview.net.

Bernardini, Sara and Christian Muise, eds. (2024). Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling (ICAPS 2024). AAAI Press.

Cai, Jin-Yi, Martin Fürer, and Neil Immerman (1992). **"An optimal lower bound on the number of variables for graph identification".** In: *Combinatorica* 12.4, pp. 389–410.

Chen, Dillon Z., Felipe Trevizan, and Sylvie Thiébaux (2024). **"Return to Tradition: Learning Reliable Heuristics with Classical Machine Learning".** In: *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling (ICAPS 2024)*. Ed. by Sara Bernardini and Christian Muise. AAAI Press, pp. 68–76.

Demir, Caglar and Axel-Cyrille Ngonga Ngomo (2023). **"Neuro-Symbolic Class Expression Learning".** In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*. Ed. by Edith Elkind. International Joint Conferences on Artificial Intelligence Organization, pp. 3624–3632.

Drexler, Dominik, Simon Ståhlberg, Blai Bonet, and Hector Geffner (2024). **"Symmetries and Expressive Requirements for Learning General Policies".** In: *Proceedings of the Twenty-First International Conference on Principles of Knowledge Representation and Reasoning (KR 2024)*. Ed. by Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco. IJCAI Organization.

Francès, Guillem, Blai Bonet, and Hector Geffner (2021). **"Learning General Planning Policies from Small Examples Without Supervision".** In: *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI 2021)*. Ed. by Kevin Leyton-Brown and Mausam. AAAI Press, pp. 11801–11808.

Francès, Guillem, Augusto B. Corrêa, Cedric Geissmann, and Florian Pommerening (2019). **"Generalized Potential Heuristics for Classical Planning".** In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI 2019)*. Ed. by Sarit Kraus. IJCAI, pp. 5554–5561.

Grohe, Martin (2021). **"The Logic of Graph Neural Networks".** In: *Proceedings of the Thirty-Sixth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*, pp. 1–17.

Horcík, Rostislav and Gustav Sír (2024). **"Expressiveness of Graph Neural Networks in Planning Domains".** In: *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling (ICAPS 2024)*. Ed. by Sara Bernardini and Christian Muise. AAAI Press, pp. 281–289.

Ståhlberg, Simon, Blai Bonet, and Hector Geffner (2023). **"Learning General Policies with Policy Gradient Methods".** In: *Proceedings of the Twentieth International Conference on Principles of Knowledge Representation and Reasoning (KR 2023)*. Ed. by Pierre Marquis, Tran Cao Son, and Gabriele Kern-Isberner. IJCAI Organization, pp. 647–657.

Weisfeiler, Boris and Andrei Leman (1968). **"The Reduction of a Graph to a Canonical Form and an Algebra Arising During This Reduction.".** In: *Nauchno-Technicheskaya Informatsia*.

Xu, Keyulu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka (2018). **"How Powerful are Graph Neural Networks?"** In: *ArXiv* abs/1810.00826. URL: https://api.semanticscholar.org/CorpusID:52895589.

Appendix 1: Generalized Policies Definition

Definition (State trajectory)

A **state trajectory** is a path in the state model $\mathcal{S}_P$ of a task $P \in \mathcal{Q}$, i.e., a sequence of states $s_1, s_2, \ldots, s_n$ such that, for every $i = 1, \ldots, n-1$, there is some action a_i with $(s_i, s_{i+1}) \in T_P$.

Definition (Policy)

A **policy** π for a class of tasks $\mathcal{Q}$ is a relation that, for every task $P \in \mathcal{Q}$, defines $\pi_P \subseteq T_P$. We say that (s, s') is π**-compatible** iff $(s, s') \in \pi_P$.

Definition (Policy-Compatible Trajectory)

A state trajectory $s_0, s_1, \ldots$ is π**-compatible** iff every transition (s_i, s_{i+1}) is π-compatible. A π-compatible state trajectory is **maximal** if it either ends in a goal state G_P or it cannot be extended by adding more states while remaining π-compatible.

Definition (Generalized Policy)

We say that a policy π is a **generalized policy** for $\mathcal{Q}$ if, for every task $P \in \mathcal{Q}$, every maximal π-compatible trajectory starting at $s_0 = I_P$ has polynomial length and ends in a goal state in G_P.

Appendix 2: A Planning Example for 1-WL

- Example from Blocks: color refinement cannot separate two states with different optimal values.

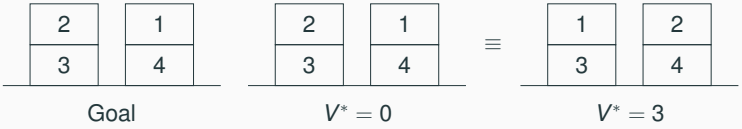


Figure 5: Color refinement cannot distinguish two Blocks states.